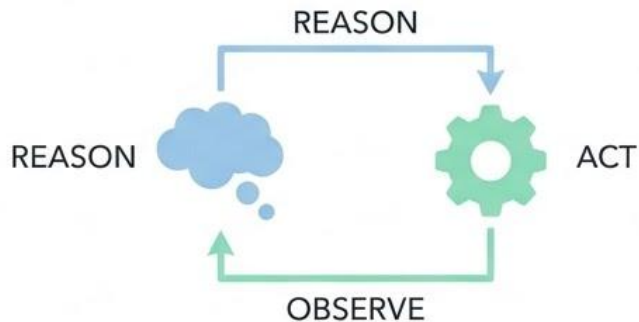


The ReAct Pattern & LangChain Agents

From Tools to Intelligent Reasoning Agents



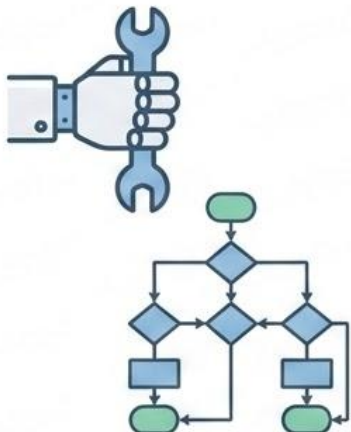
Topics to be covered

- ReAct Pattern Explained
- Building LangChain Agents
- Advanced Techniques
- Practical Workflows



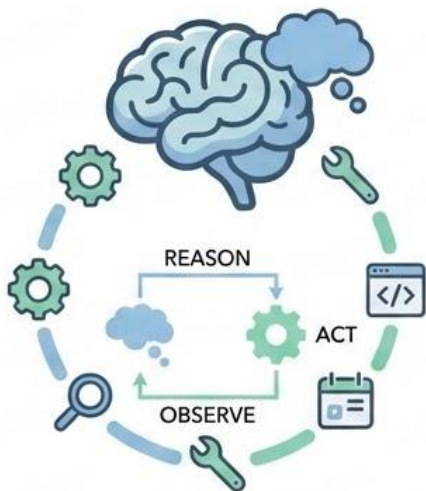
Why Agents Need Reasoning

SIMPLE TOOL-USER



Fixed, single-step execution.

INTELLIGENT REASONING AGENT

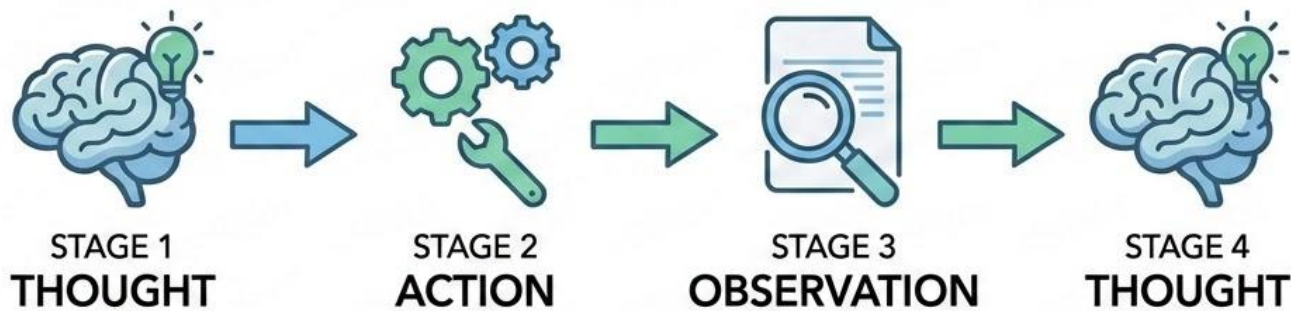


Autonomous planning & multi-step orchestration.

- Understanding User Intent & Ambiguity
- Long-Horizon Goal Decomposition
- Adaptive Planning & Error Recovery
- Synthesizing Information Across Sources
- Self-Reflection for Continuous Improvement

The ReAct Pattern Explained

ReAct Loop



Simple Explanation

Agents iterate through cocceating reasoning and action, extercing action or eill informations.



Real-World Example

Booking a complex travel itinerary
(Search -> Book -> Observe Error
Error -> Adapt Plan)

ReAct Loop in Detail



Small Example Trace

[THOUGHT]: need to find the population of Tokyo.

[ACTION]: <search_tool> "Tokyo population"

[OBSERVATION]: Output: 37,194,104

[THOUGHT]: I now know the population. I will provide the answer.

[ANSWER]: The population of Tokyo is 37.19 million.

Thinking Models vs Normal Models

1

Normal Models



Direct Input/Output
Fast, single-step response



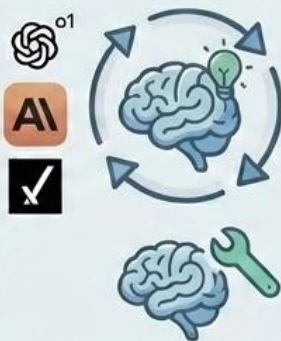
Stateless Interactions
Limited context retention



Example: Simple Q&A
Fixed knowledge retrieval

2

Thinking Models



Recursive Reasoning Loop
Internal multi-step planning

Long-Running Thought
Recursive thought over time

Adaptable Plan
Dynamic goal adjustment

Key Attributes of Thinking Models for ReAct



Improved Problem Solving
Higher task success on complex queries

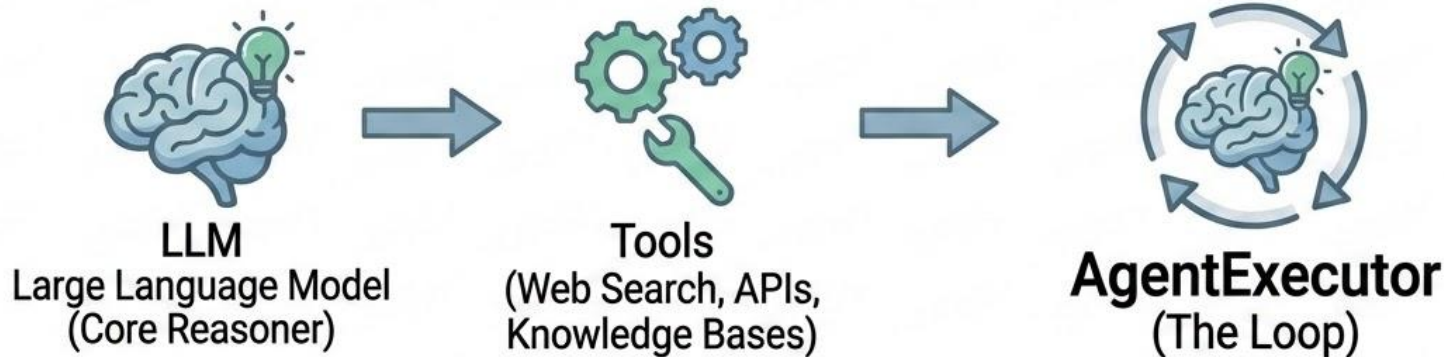


Robust Error Recovery
Recognizes and adapts from failures



Enhanced Task Execution
More reliable multi-tool use

LangChain Agents Basics



Core Components explained

- **AgentExecutor:** Manages the multi-step reasoning loop (Thought, Action, Observation).
- **LLM (Brain):** The logic engine that decides the best action or tool to use.
- **Tools:** External interfaces the LLM accesses (e.g., search, calculators).
- **Integration:** Connects the LLM's language capability with external actions.
- **Feedback Loop:** Constantly updates the plan based on new information.
- **Dynamic Planning:** Adjusts goals and steps as the task executes.

Create Agents with create_agent

Simple example demonstrating the core function call with key parameters.

```
from langchain.agents import create_agent
from langchain.llms import OpenAI
from langchain.tools import (search_tool, calculator_tool, weather_tool)
from langchain.memory import ConversationBufferMemory
from langchain.middleware import logging_middleware
```

```
# Initialize model, tools, memory, middleware
llm = OpenAI(temperature=0)
tools = [search_tool, calculator_tool, weather_tool]
memory = ConversationBufferMemory()
middleware = [logging_middleware]
```

```
# Create the agent with specified components
agent = create_agent(
    model=llm,
    tools=tools,
    memory=memory,
    middleware=middleware
)
```

Key Parameters

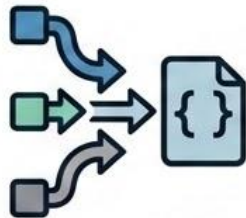
- **model:** Specifies the underlying Large Language Model (e.g., GPT-3.5-turbo, Gemini).
- **tools:** Lists the external interfaces (search, APIs, knowledge bases) available to the agent.
- **memory:** Retains conversation history and user context for personalized, ongoing interactions.
- **middleware:** Manages the agent execution flow, adding logging, authentication, and error handling.

Best Practices for Agents



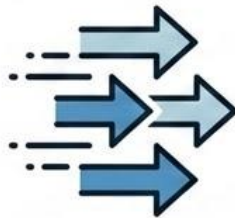
Clear Tools

- Be explicit about tool purpose and limits.



Structured Outputs

- Enforce defined data formats for reliability.



Parallel Execution

- Optimize multi-tasking and latency.



Memory Management

- Implement robust conversation context storage.

LangGraph vs Classic Agents

Classic Agents (Single Loop)

Simple, linear execution path.
Limited structural control.
Ideal for straightforward tasks.

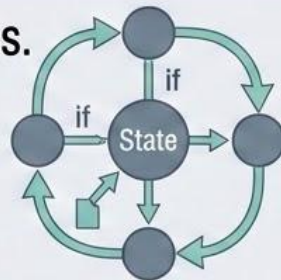


WHEN TO USE LANGGRAPH FOR MORE CONTROL:

- Multi-step, interdependent workflows.
- Required state management across agent steps.
- Conditional logic & branching based on tool output.
- Dynamic planning that needs memory.
- Creating recursive and cyclic loops.

LangGraph (Structured Control)

Explicit flow control via graphs.
Shared State management.
Cyclic structures supported.



- Multi-step, interdependent workflows.
- Required state management across agent steps.
- Conditional logic & branching based on tool output.
- Dynamic planning that needs memory.
- Creating recursive and cyclic loops.

Common Pitfalls & Real-World Tips

11

Common Pitfalls & Solutions

- Inconsistent tool responses -->
 - Solution: Validate outputs and handle edge cases.
- Agent looping indefinitely -->
 - Solution: Implement a maximum step limit in middleware.
- Memory overload with long context -->
 - Solution: Use conversational memory strategies (e.g., summary, window).

Practical Tips for Production Agents

- Set explicit limits for tool execution time.
- Monitor agent step and tool invocation costs.
- Use a managed LLM API (e.g., robust rate limiting).
- Implement thorough integration testing.
- Continuously monitor for drift and errors.

Summary

- Classic Agents are optimal for simple, linear task execution.
 - LangGraph provides structured control and shared state management for complex agent workflows.
 - LangGraph enables advanced conditional logic, branching, and cyclical operations.
 - For production-ready agents, implement strict input/output validation.
 - Always set maximum step limits to prevent indefinite loops.
 - Utilize advanced memory management strategies (e.g., summary, sliding windows).
- 